

NIST FIPS 197

Advanced Encryption Standard (AES)

한국어 학습 번역본

원문: NIST FIPS 197-upd1 (2023-05-09 업데이트)

학습 주석 | 이 문서는 원문의 의미와 구조를 따라 번역한 학습용 자료입니다. AES, State, S-box, Round Key, Key Schedule, SubBytes, ShiftRows, MixColumns, AddRoundKey, Cipher, Inverse Cipher 같은 기술 명칭은 구현 문서 및 코드와 대응하기 쉽도록 English 를 유지하고, 바로 뒤에 한국어 뜻과 설명을 붙였습니다. 수식·hex 값·pseudocode 의 식별자는 원문 표기를 유지합니다.

학습 주석 | 특히 inverse 는 '역변환/역연산'이라는 뜻입니다. 어떤 변환을 적용한 결과에 inverse 를 적용하면 원래 값으로 돌아옵니다. 예: SHIFTROWS()의 inverse 는 INVSHIFTROWS()입니다.

발간 정보와 요약

문서명: Federal Information Processing Standards Publication 197 - Advanced Encryption Standard (AES)

분류: Computer Security / Cryptography

발행: 2001-11-26, 업데이트: 2023-05-09

기관: National Institute of Standards and Technology (NIST), Information Technology Laboratory

Abstract - 초록

2000 년 NIST 는 Advanced Encryption Standard(AES) 공모의 승자로 Rijndael block cipher family 를 선정했다. Block cipher 는 많은 cryptographic service 의 기반이며, 특히 data confidentiality(데이터 기밀성)를 보장하는 데 핵심적으로 사용된다.

이 표준은 Rijndael 계열 중 AES-128, AES-192, AES-256 세 가지를 규정한다. 세 알고리즘 모두 128-bit block 을 변환하며, 이름 뒤의 숫자 128/192/256 은 사용되는 cryptographic key 의 bit 길이를 의미한다.

confidentiality : 기밀성 - 권한 없는 사람이 데이터 내용을 알아볼 수 없게 보호하는 성질

block cipher : 블록 암호 - 고정 길이 데이터 block 을 key 에 따라 다른 block 으로 치환하는 암호 알고리즘

Rijndael : 레인달 - AES 공모에서 선정된 원래 block cipher family 의 이름

FIPS 197 공표 사항

1. 표준명: Advanced Encryption Standard (AES), FIPS 197.
2. 표준 분류: Computer Security Standard, Cryptography.
3. AES는 전자 데이터를 보호하기 위해 사용할 수 있는 FIPS-approved cryptographic algorithm이다. Symmetric block cipher이며 digital information의 encrypt/decrypt를 수행한다.
4. AES의 key 길이는 128, 192, 256 bit이며 block 크기는 항상 128 bit이다.
5. 승인 기관: U.S. Secretary of Commerce.
6. 관리 기관: NIST Information Technology Laboratory.
7. 구현은 software, firmware, hardware 또는 이들의 조합으로 가능하다.
8. AES는 FIPS-approved 또는 NIST-recommended mode of operation과 함께 사용해야 한다.
9. NIST는 구현이 표준과 일치하는지 확인하기 위한 validation program을 운영한다.
10. FIPS 197은 2002-05-26부터 효력이 발생했다.
11. NIST는 암호 분석과 기술 발전을 반영하여 AES 표준을 계속 재검토한다.

학습 주석 | AES 그 자체는 '128-bit block 하나를 key로 변환하는 primitive'입니다. 저장장치 암호화에 AES-XTS를 쓰려면 AES에 XTS mode가 추가됩니다. 즉 AES와 AES-XTS는 같은 말이 아닙니다.

학습 목차

- 1. Introduction - AES 가 무엇인지
- 2. Definitions - 핵심 용어와 함수
- 3. Notation and Conventions - bit/byte/word/State 배치 규칙
- 4. Mathematical Preliminaries - $GF(2^8)$ 연산
- 5. Algorithm Specifications - CIPHER, SUBBYTES, SHIFTRROWS, MIXCOLUMNS, ADDROUNDKEY, KEYEXPANSION
- 5.3 Inverse - 복호화 방향의 역변환
- 6. Implementation Considerations - 실제 software/FPGA 구현 시 고려사항
- Appendix A - Key Expansion 예제
- Appendix B - AES-128 한 block 전체 round 예제
- Appendix C - Example Vector
- Appendix D - 2023 update 변경사항

1. Introduction

Block 은 주어진 고정 길이의 bit sequence 이다. Block cipher 는 key 라고 부르는 bit sequence 에 의해 parameterized 되는 block permutation 의 family 이다.

1997 년 NIST 는 AES 개발 절차를 시작하고 공개적으로 block cipher 후보 알고리즘을 모집했다. 2000 년 NIST 는 Rijndael 을 AES 로 선정했다.

FIPS 197 이 규정하는 Rijndael 의 세 가지 instance 는 AES-128, AES-192, AES-256 이다. suffix 는 key 길이를 의미 하며, 세 경우 모두 block size 는 128 bit 이다. Rijndael 자체는 다른 block size 와 key length 도 지원하지만, 그러한 조합은 이 FIPS 197 의 AES 가 아니다.

Algorithm	Key length	Block size	Rounds
AES-128	128 bit	128 bit	10
AES-192	192 bit	128 bit	12
AES-256	256 bit	128 bit	14

학습 주석 | AES-256 은 block 이 256-bit 라는 뜻이 아닙니다. AES-256 도 block 은 128-bit 이고 key 만 256-bit 입니다.

2. Definitions

2.1 Terms and Acronyms

AES : Advanced Encryption Standard - NIST 가 표준화한 symmetric block cipher.

Affine transformation : 아핀 변환 - matrix multiplication 뒤에 vector addition 이 이어지는 변환.

Array : 배열 - 정수 index 로 각 element 를 식별하는 고정 크기 data structure.

Bit : 비트 - 0 또는 1 의 binary digit.

Block : 블록 - 고정 길이 bit sequence. AES 에서는 128 bit.

Block cipher : 블록 암호 - key 가 어떤 permutation 을 사용할지 결정하는 block permutation family.

Byte : 바이트 - 8 bit sequence.

Equivalent inverse cipher : 등가 역 cipher - CIPHER()와 유사한 구조를 갖도록 표현한 inverse cipher. 수정된 key schedule 을 사용.

Key : 키 - block cipher family 중 어떤 permutation 을 사용할지 결정하는 parameter.

Key schedule : 키 스케줄 - KEYEXPANSION()으로 생성되는 round key sequence.

Rijndael : 레인달 - AES 공모에서 선정된 block cipher.

Round : 라운드 - State 에 반복 적용되는 transformation sequence.

Round key : 라운드 키 - 원래 key 로부터 파생된 $Nr+1$ 개의 128-bit key 중 하나.

State : 상태 배열 - AES 처리 중간값을 나타내는 4x4 byte array.

S-box : Substitution box, 치환표 - SUBBYTES()와 KEYEXPANSION()에서 byte 를 다른 byte 로 1:1 치환하는 비선형 table.

Word : 워드 - 32 bit = 4 byte 묶음.

2.2 List of Functions

ADDROUNDKEY() : State 와 Round Key 를 bitwise XOR 로 결합한다.

AES-128()/AES-192()/AES-256() : 각 key 길이에 대응하는 AES block cipher instance.

CIPHER() : AES forward transformation. 일반적으로 encryption 방향으로 이해하면 된다.

INVCIPHER() : CIPHER()의 inverse. 일반적으로 decryption 방향.

EQINVCIPHER() : modified key schedule 을 사용하는 equivalent inverse cipher.

MIXCOLUMNS() : State 의 각 column 내부 byte 를 $GF(2^8)$ matrix 연산으로 섞는다.

INVMIXCOLUMNS() : MIXCOLUMNS()의 inverse.

SBOX()/INVSBOX() : S-box 치환 / 그 역치환.

SHIFTROWS()/INVSHIFTROWS() : State row 의 cyclic shift / 역방향 shift.

SUBBYTES()/INVSUBBYTES() : 각 State byte 에 S-box / inverse S-box 적용.

KEYEXPANSION() : 원래 key 로부터 모든 Round Key 를 만든다.

KEYEXPANSIONEIC() : Equivalent Inverse Cipher 용 modified round keys 생성.

ROTWORD() : 4-byte word 를 1 byte 왼쪽 cyclic rotation.

SUBWORD() : word 의 4 개 byte 각각에 S-box 적용.

XTIMES() : $GF(2^8)$ 에서 입력 byte 에 polynomial x , 즉 {02}를 곱한다.

2.3 Algorithm Parameters and Symbols

$GF(2)$: element 가 0,1 두 개인 finite field.

$GF(2^8)$: 256 개 element 를 갖는 finite field. AES 의 byte arithmetic 에 사용.

in : CIPHER()/INVCIPHER() 입력 16-byte array.

out : CIPHER()/INVCIPHER() 출력 16-byte array.

key : AES key 를 구성하는 N_k 개의 32-bit word array.

N_b : State 를 구성하는 32-bit word 의 개수. FIPS 197 AES 에서는 항상 4.

N_k : Key 를 구성하는 32-bit word 의 개수. AES-128=4, AES-192=6, AES-256=8.

N_r : Round 수. AES-128=10, AES-192=12, AES-256=14.

Rcon : Round Constant word array.

state : $4 \times 4 = 16$ -byte 2 차원 State array.

w : Key Schedule word array.

$\oplus$: XOR. bit/byte/word 단위 exclusive-OR.

$\cdot$: $GF(2^8)$ multiplication.

$*$: integer multiplication.

$\leftarrow$: pseudocode 에서 assignment.

3. Notation and Conventions

3.1 Inputs and Outputs

AES block cipher의 data input 과 output 은 모두 128-bit block 이다. Key 는 보통 여러 번의 block cipher invocation 에 걸쳐 유지되는 bit sequence 이다. AES-128, AES-192, AES-256 의 key 길이는 각각 128, 192, 256 bit 이다.

3.2 Bytes

AES 의 기본 processing unit 은 byte, 즉 8-bit sequence 이다. 예를 들어 {10100011}은 hexadecimal 로 {a3}이다. 한 byte 의 bit index 는 왼쪽에서 오른쪽으로 b7, b6, ..., b0 순서로 감소한다.

4-bit	hex	4-bit	hex	4-bit	hex	4-bit	hex
0000	0	0100	4	1000	8	1100	c
0001	1	0101	5	1001	9	1101	d
0010	2	0110	6	1010	a	1110	e
0011	3	0111	7	1011	b	1111	f

3.3 Indexing of Byte Sequences

8k bit sequence $r_0, r_1, \dots, r_{(8k-1)}$ 이 있을 때 j 번째 byte a_j 는 $r(8j)$ 부터 $r(8j+7)$ 까지의 8 bit 로 정의한다. 따라서 128-bit block 은 $a_0 \dots a_{15}$ 의 16 byte 로 표현된다.

학습 주석 | 여기서 헷갈리기 쉬운 점은 '전체 sequence 의 bit index'와 'byte 내부 bit index'가 서로 다른 규칙을 쓴다는 것입니다. 전체 sequence 는 0 부터 증가하지만, 한 byte 안에서는 왼쪽 MSB 가 bit 7 입니다.

3.4 The State

AES 내부 연산은 16 byte 를 4x4 2 차원 배열로 배치한 State 에서 수행한다. State byte 는 $s[r,c]$ 로 표기하며 r 은 row, c 는 column 이다.

입력 byte 배치:

$$s[r,c] = in[r + 4c] \quad (0 \leq r < 4, 0 \leq c < 4)$$

즉:

in0	in4	in8	in12
in1	in5	in9	in13
in2	in6	in10	in14
in3	in7	in11	in15

학습 주석 | AES State 는 흔히 처음 보는 사람이 row-major 로 오해합니다. 실제로 입력은 column 부터 채워집니다.

RTL/FPGA 구현에서 byte ordering 이 틀리면 모든 계산은 맞아도 최종 ciphertext 가 NIST vector 와 일치하지 않습니다.

3.5 Arrays of Words

Word 는 4 byte = 32 bit 이다. State 의 각 column 하나가 word 하나로 해석된다. 따라서 AES block 하나는 4 개의 word 로 이루어진다.

4. Mathematical Preliminaries

AES의 일부 transformation에서는 State의 각 byte를 finite field, 즉 Galois Field $GF(2^8)$ 의 256개 element 중 하나로 해석한다.

byte {b7 b6 ... b0}는 다음 polynomial로 해석한다:

$$b(x) = b7 \cdot x^7 + b6 \cdot x^6 + \dots + b1 \cdot x + b0$$

예: {01100011} = $x^6 + x^5 + x + 1$

4.1 Addition in $GF(2^8)$

$GF(2^8)$ 의 addition은 polynomial coefficient를 modulo 2로 더하는 것이다. bit 관점에서는 XOR와 완전히 같다.

$$1 \text{ XOR } 1 = 0$$

$$1 \text{ XOR } 0 = 1$$

$$0 \text{ XOR } 0 = 0$$

$$\{57\} \text{ XOR } \{83\} = \{d4\}$$

학습 주석 | AES에서 '+'와 '-'가 일반 정수 산술처럼 동작한다고 생각하면 안 됩니다. $GF(2)$ 에서는 $1+1=0$ 이므로 addition과 subtraction이 동일합니다.

4.2 Multiplication in $GF(2^8)$

$GF(2^8)$ multiplication은 두 byte를 polynomial로 곱한 뒤, 다음 irreducible polynomial로 modulo reduction한다.

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

특히 {02}를 곱하는 연산을 $XTIMES()$ 로 정의한다. 최상위 bit b7이 0이면 왼쪽 shift, b7이 1이면 왼쪽 shift 후 {1b}를 XOR한다.

$$XTIMES(\{57\}) = \{ae\}$$

$$XTIMES(\{ae\}) = \{47\}$$

$$XTIMES(\{47\}) = \{8e\}$$

$$XTIMES(\{8e\}) = \{07\}$$

원문의 예: $\{57\} \cdot \{13\} = \{fe\}$

4.3 Multiplication of Words by a Fixed Matrix

MIXCOLUMNS()와 INVMIXCOLUMNS()는 4-byte word 에 fixed matrix 를 곱하는 형태로 정의된다. 각 곱셈과 덧셈은 일반 정수 산술이 아니라 $GF(2^8)$ arithmetic 이다.

```
[d0]   [a0 a3 a2 a1] [b0]
[d1] = [a1 a0 a3 a2] [b1]
[d2]   [a2 a1 a0 a3] [b2]
[d3]   [a3 a2 a1 a0] [b3]
```

4.4 Multiplicative Inverses in $GF(2^8)$

0 이 아닌 byte b 의 multiplicative inverse b^{-1} 은 다음을 만족하는 유일한 byte 이다.

$$b \cdot b^{-1} = \{01\}$$

AES 의 SUBBYTES()는 이 multiplicative inverse 를 사용한다. 원문은 $b^{-1} = b^{254}$ 로 계산할 수 있음을 제시하고, 또는 extended Euclidean algorithm 으로 구할 수 있다고 설명한다.

학습 주석 | inverse 가 여기서는 '역수' 의미입니다. 예를 들어 일반 실수에서 5 의 multiplicative inverse 는 $1/5$ 이고, $GF(2^8)$ 에서는 해당 field 연산 기준으로 곱했을 때 $\{01\}$ 이 되는 byte 를 말합니다.

5. Algorithm Specifications

AES-128, AES-192, AES-256 의 forward function 은 CIPHER(), 그 inverse 는 INVCIPHER()로 표기한다. 핵심은 State 에 여러 round 의 fixed transformation 을 반복 적용하는 것이다.

항목	AES-128	AES-192	AES-256
Nk (32-bit words)	4	6	8
Key length	128 bit	192 bit	256 bit
Nb	4	4	4
Block size	128 bit	128 bit	128 bit
Nr	10	12	14

AES-128(in,key) = CIPHER(in, 10, KEYEXPANSION(key))

AES-192(in,key) = CIPHER(in, 12, KEYEXPANSION(key))

AES-256(in,key) = CIPHER(in, 14, KEYEXPANSION(key))

학습 주석 | Round 수가 key 길이에 따라 달라집니다. AES-128 은 10 round, AES-256 은 14 round 입니다.

5.1 CIPHER()

한 round 를 구성하는 네 transformation 은 다음과 같다.

12. SUBBYTES(): State 의 각 byte 를 S-box 로 치환한다.
13. SHIFTRROWS(): State 의 각 row 를 서로 다른 offset 만큼 cyclic shift 한다.
14. MIXCOLUMNS(): 각 column 내부 4 byte 를 $GF(2^8)$ matrix 연산으로 섞는다.
15. ADDROUNDKEY(): State 와 해당 Round Key 를 XOR 한다.

초기에는 ADDROUNDKEY()만 한 번 수행하고, middle round 에서는 네 transformation 을 모두 수행하며, 마지막 round 에서는 MIXCOLUMNS()이 빠진다.

```
procedure CIPHER(in, Nr, w)
  state <- in
  state <- ADDROUNDKEY(state, w[0..3])

  for round = 1 to Nr-1
    state <- SUBBYTES(state)
    state <- SHIFTRROWS(state)
    state <- MIXCOLUMNS(state)
    state <- ADDROUNDKEY(state, w[4*round .. 4*round+3])

  state <- SUBBYTES(state)
  state <- SHIFTRROWS(state)
  state <- ADDROUNDKEY(state, w[4*Nr .. 4*Nr+3])

  return state
end procedure
```

학습 주석 | Final round 에 MIXCOLUMNS()이 없다는 점은 매우 중요합니다. AES RTL 구현에서 자주 발생하는 오류 중 하나입니다.

5.1.1 SUBBYTES()

SUBBYTES()는 invertible(역변환 가능)하고 non-linear(비선형)인 transformation 이다. State 의 16 개 byte 각각에 동일한 AES S-box 를 독립적으로 적용한다.

S-box 는 두 단계로 구성된다. 먼저 입력 byte 가 {00}이 아니면 $GF(2^8)$ 의 multiplicative inverse 를 구하고, 그 결과에 affine transformation 을 적용한다. 입력이 {00}이면 중간값은 {00}으로 정의한다.

S-box lookup 예: 입력이 {53}이면 Table 4 에서 row 5, column 3 을 찾아 출력 {ed}를 얻는다.

학습 주석 | FPGA 구현에서는 S-box 를 LUT/ROM 형태로 구현하거나, composite field arithmetic 으로 계산할 수 있습니다. 하지만 bit-exact 검증이 목적이라면 우선 표 그대로 lookup 하는 방식이 이해하기 쉽습니다.

x	y=0..f (16 개 값)
0	63 7c 77 7b f2 6b 6f c5 30 01 67 2b fe d7 ab 76
1	ca 82 c9 7d fa 59 47 f0 ad d4 a2 af 9c a4 72 c0
2	b7 fd 93 26 36 3f f7 cc 34 a5 e5 f1 71 d8 31 15
3	04 c7 23 c3 18 96 05 9a 07 12 80 e2 eb 27 b2 75
4	09 83 2c 1a 1b 6e 5a a0 52 3b d6 b3 29 e3 2f 84
5	53 d1 00 ed 20 fc b1 5b 6a cb be 39 4a 4c 58 cf
6	d0 ef aa fb 43 4d 33 85 45 f9 02 7f 50 3c 9f a8
7	51 a3 40 8f 92 9d 38 f5 bc b6 da 21 10 ff f3 d2
8	cd 0c 13 ec 5f 97 44 17 c4 a7 7e 3d 64 5d 19 73
9	60 81 4f dc 22 2a 90 88 46 ee b8 14 de 5e 0b db
a	e0 32 3a 0a 49 06 24 5c c2 d3 ac 62 91 95 e4 79
b	e7 c8 37 6d 8d d5 4e a9 6c 56 f4 ea 65 7a ae 08
c	ba 78 25 2e 1c a6 b4 c6 e8 dd 74 1f 4b bd 8b 8a
d	70 3e b5 66 48 03 f6 0e 61 35 57 b9 86 c1 1d 9e
e	e1 f8 98 11 69 d9 8e 94 9b 1e 87 e9 ce 55 28 df
f	8c a1 89 0d bf e6 42 68 41 99 2d 0f b0 54 bb 16

5.1.2 SHIFTRROWS()

SHIFTRROWS()는 State 의 마지막 세 row 를 cyclic shift 한다. row index r 만큼 왼쪽으로 이동한다.

```
row 0: 0 byte shift
row 1: 1 byte left rotate
row 2: 2 byte left rotate
row 3: 3 byte left rotate
```

$$s'[r,c] = s[r,(c+r) \bmod 4]$$

Before:

```
a0 a1 a2 a3
b0 b1 b2 b3
c0 c1 c2 c3
d0 d1 d2 d3
```

After:

```
a0 a1 a2 a3
b1 b2 b3 b0
c2 c3 c0 c1
d3 d0 d1 d2
```

5.1.3 MIXCOLUMNS()

MIXCOLUMNS()는 State 의 각 column 을 독립적으로 fixed matrix 와 곱한다.

```
[s'0,c]   [02 03 01 01] [s0,c]
[s'1,c] = [01 02 03 01] [s1,c]
[s'2,c]   [01 01 02 03] [s2,c]
[s'3,c]   [03 01 01 02] [s3,c]
```

모든 multiplication 과 XOR 는 $GF(2^8)$ 규칙을 따른다.

5.1.4 ADDROUNDKEY()

ADDROUNDKEY()는 State 의 각 column 과 Key Schedule 의 대응 word 를 bitwise XOR 한다.

$$[s'0,c,s'1,c,s'2,c,s'3,c] = [s0,c,s1,c,s2,c,s3,c] \text{ XOR } w[4*\text{round}+c]$$

학습 주석 | ADDROUNDKEY()는 XOR 이므로 자기 자신이 inverse 입니다. 같은 Round Key 를 다시 XOR 하면 원래 값으로 돌아갑니다.

5.2 KEYEXPANSION()

KEYEXPANSION()은 원래 AES key 를 입력받아 $4 \cdot (Nr+1)$ 개의 32-bit word 를 생성한다. ADDROUNDKEY()가 초기 1 회 + Nr 회, 총 Nr+1 회 호출되므로 round 마다 4 word 가 필요하기 때문이다.

KEYEXPANSION()은 ROTWORD(), SUBWORD(), Rcon 을 사용한다.

$ROTWORD([a_0, a_1, a_2, a_3]) = [a_1, a_2, a_3, a_0]$

$SUBWORD([a_0, a_1, a_2, a_3]) = [SB0X(a_0), SB0X(a_1), SB0X(a_2), SB0X(a_3)]$

j	Rcon[j]	j	Rcon[j]
1	01 00 00 00	6	20 00 00 00
2	02 00 00 00	7	40 00 00 00
3	04 00 00 00	8	80 00 00 00
4	08 00 00 00	9	1b 00 00 00
5	10 00 00 00	10	36 00 00 00

Key expansion recurrence:

- i 가 Nk 의 배수이면: $w[i] = w[i-Nk] \text{ XOR } SUBWORD(ROTWORD(w[i-1])) \text{ XOR } Rcon[i/Nk]$
- AES-256에서 $i \bmod 8 = 4$ 이면: $w[i] = w[i-Nk] \text{ XOR } SUBWORD(w[i-1])$
- 그 외: $w[i] = w[i-Nk] \text{ XOR } w[i-1]$

```
procedure KEYEXPANSION(key)
```

```
  i <- 0
```

```
  while i <= Nk-1
```

```
    w[i] <- key[4*i .. 4*i+3]
```

```
    i <- i+1
```

```
  while i <= 4*Nr+3
```

```
    temp <- w[i-1]
```

```
    if i mod Nk = 0
```

```
      temp <- SUBWORD(ROTWORD(temp)) XOR Rcon[i/Nk]
```

```
    else if Nk > 6 and i mod Nk = 4
```

```
      temp <- SUBWORD(temp)
```

```
    w[i] <- w[i-Nk] XOR temp
```

```
    i <- i+1
```

```
  return w
```

```
end procedure
```

학습 주석 | AES-128의 경우 44 word = 176 byte 의 key schedule 이 생성됩니다. 원래 key 16 byte 가 11 개의 16-byte Round Key로 확장되는 셈입니다.

5.3 INVCIPHER()

INVCIPHER()는 CIPHER()에서 사용한 transformation 을 inverse 로 바꾸고 순서를 역으로 실행한다.

```
procedure INVCIPHER(in, Nr, w)
  state <- in
  state <- ADDROUNDKEY(state, w[4*Nr .. 4*Nr+3])

  for round = Nr-1 downto 1
    state <- INVSHIFTRROWS(state)
    state <- INVSUBBYTES(state)
    state <- ADDROUNDKEY(state, w[4*round .. 4*round+3])
    state <- INVMIXCOLUMNS(state)

  state <- INVSHIFTRROWS(state)
  state <- INVSUBBYTES(state)
  state <- ADDROUNDKEY(state, w[0..3])

  return state
end procedure
```

5.3.1 INVSHIFTRROWS()

SHIFTRROWS()의 inverse 이다. row index r 만큼 오른쪽으로 cyclic shift 한다.

$$s'[r,c] = s[r,(c-r) \bmod 4]$$

5.3.2 INVSUBBYTES()

SUBBYTES()의 inverse 이다. 각 byte 에 INVSBOX()를 적용한다. INVSBOX 는 SBOX 의 input/output 관계를 뒤집은 1:1 lookup table 이다.

5.3.3 INVMIXCOLUMNS()

MIXCOLUMNS()의 inverse 이며 다음 fixed matrix 를 사용한다.

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$

5.3.4 Inverse of ADDROUNDKEY()

ADDROUNDKEY()는 자기 자신이 inverse 이다. $A \oplus K \oplus K = A$ 이기 때문이다.

5.3.5 EQINVCIPHER()

AES의 algebraic property를 이용하면 CIPHER()와 유사한 round structure를 가진 equivalent inverse cipher를 만들 수 있다. 이를 EQINVCIPHER()라고 한다.

EQINVCIPHER()는 수정된 key schedule dw를 사용하며, KEYEXPANSIONEIC()가 이 dw를 생성한다. 첫 번째와 마지막 Round Key는 원래 w와 같고, 중간 Round Key에는 INVMIXCOLUMNS()를 적용한다.

```
procedure EQINVCIPHER(in, Nr, dw)
  state <- in
  state <- ADDROUNDKEY(state, dw[4*Nr .. 4*Nr+3])
  for round = Nr-1 downto 1
    state <- INVSUBBYTES(state)
    state <- INVSHIFTRROWS(state)
    state <- INVMIXCOLUMNS(state)
    state <- ADDROUNDKEY(state, dw[4*round .. 4*round+3])
  state <- INVSUBBYTES(state)
  state <- INVSHIFTRROWS(state)
  state <- ADDROUNDKEY(state, dw[0..3])
  return state
end procedure
```

학습 주석 | 소프트웨어나 FPGA에서 encryption/decryption datapath를 얼마나 공유할지 설계할 때 equivalent inverse cipher 구조가 도움이 될 수 있습니다. 다만 처음 학습할 때는 표준 INVCIPHER() 순서를 먼저 정확히 이해하는 것이 좋습니다.

6. Implementation Considerations

6.1 Key Length Requirements

AES 구현은 128, 192, 256 bit key length 중 적어도 하나를 지원해야 한다. 둘 또는 세 종류 모두를 지원할 수도 있다.

6.2 Keying Restrictions

Cryptographic key 가 적절한 방법으로 생성되었다면, AES 에 사용하는 결과 key 에 별도의 추가 제한을 두지 않는다. 원문은 NIST SP 800-133 Rev.2 의 key generation guideline 을 참조한다.

6.3 Parameter Extensions

현 FIPS 197 은 N_k , N_b , N_r 의 허용값을 명확하게 규정한다. 미래 개정에서 값이 추가될 가능성을 고려해 구현을 유연하게 설계할 수 있으나, 현재 표준에 conform 하는 AES 조합은 지정된 세 조합뿐이다.

6.4 Implementation Suggestions Regarding Various Platforms

동일한 input key 와 data 에 대해 FIPS 197 이 규정한 algorithm 과 같은 output 을 낸다면 구현 방식은 달라도 equivalent implementation 으로 볼 수 있다.

하지만 빠른 구현법이 side-channel attack 에 자동으로 안전한 것은 아니다. 실제 physical implementation 은 computation time, cache behavior, fault injection 등으로 key-dependent information 을 누출할 수 있다.

side channel : 부채널 - 암호 알고리즘의 수학적 출력이 아니라 시간, 전력, cache, 전자기파 등의 물리적/구현적 정보를 통해 비밀을 추론하는 경로

cache-timing attack : 캐시 타이밍 공격 - memory/cache 접근 시간 차이로 secret-dependent access pattern 을 추론하는 공격

6.5 Modes of Operation

Block cipher mode of operation 은 AES 같은 block cipher 를 사용해 confidentiality, authentication 등 정보보호 서비스를 제공하는 cryptographic function 이다. NIST 권고 mode 는 SP 800-38 series 에 규정되어 있다.

학습 주석 | AES-XTS 는 바로 이 'mode of operation' 중 저장장치용 mode 입니다. AES primitive 위에 XTS 구조가 추가됩니다.

Appendix A - Key Expansion Examples

이 Appendix 는 각 key length 에 대해 Key Schedule 이 실제로 어떻게 생성되는지 intermediate value 를 hexadecimal 로 보여준다. Hex 값 자체는 번역 대상이 아니므로 원문 값을 그대로 유지한다.

A.1 Expansion of a 128-bit Key

Key = 2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c
Nk = 4

w0 = 2b7e1516
w1 = 28aed2a6
w2 = abf71588
w3 = 09cf4f3c

첫 확장 word w4 계산:

temp = w3 = 09cf4f3c
ROTWORD(temp) = cf4f3c09
SUBWORD(...) = 8a84eb01
Rcon[1] = 01000000
8a84eb01 XOR 01000000 = 8b84eb01
w4 = w0 XOR 8b84eb01
= 2b7e1516 XOR 8b84eb01
= a0fafe17

이후 동일한 recurrence 로 w5...w43 을 생성한다. 원문 예제의 최종 44 word key schedule 은 다음과 같다.

w00 2b7e1516	w01 28aed2a6	w02 abf71588	w03 09cf4f3c
w04 a0fafe17	w05 88542cb1	w06 23a33939	w07 2a6c7605
w08 f2c295f2	w09 7a96b943	w10 5935807a	w11 7359f67f
w12 3d80477d	w13 4716fe3e	w14 1e237e44	w15 6d7a883b
w16 ef44a541	w17 a8525b7f	w18 b671253b	w19 db0bad00
w20 d4d1c6f8	w21 7c839d87	w22 caf2b8bc	w23 11f915bc
w24 6d88a37a	w25 110b3efd	w26 dbf98641	w27 ca0093fd
w28 4e54f70e	w29 5f5fc9f3	w30 84a64fb2	w31 4ea6dc4f
w32 ead27321	w33 b58dbad2	w34 312bf560	w35 7f8d292f
w36 ac7766f3	w37 19fadc21	w38 28d12941	w39 575c006e
w40 d014f9a8	w41 c9ee2589	w42 e13f0cc8	w43 b6630ca6

학습 주석 | Round Key 0 은 w0..w3, Round Key 1 은 w4..w7, ... Round Key 10 은 w40..w43 입니다.

A.2 Expansion of a 192-bit Key

Key =

8e 73 b0 f7 da 0e 64 52 c8 10 f3 2b

80 90 79 e5 62 f8 ea d2 52 2c 6b 7b

Nk = 6

w0 = 8e73b0f7

w1 = da0e6452

w2 = c810f32b

w3 = 809079e5

w4 = 62f8ead2

w5 = 522c6b7b

AES-192 는 Nk=6 이므로 6 word 간격을 기준으로 recurrence 가 진행되며, 총 52 word 의 key schedule 을 만든다.

원문 중 핵심 확장 결과:

w06 fe0c91f7	w07 2402f5a5	w08 ec12068e
w09 6c827f6b	w10 0e7a95b9	w11 5c56fec2
w12 4db7b4bd	w13 69b54118	w14 85a74796
w15 e92538fd	w16 e75fad44	w17 bb095386
w18 485af057	w19 21efb14f	w20 a448f6d9
w21 4d6dce24	w22 aa326360	w23 113b30e6
w24 a25e7ed5	w25 83b1cf9a	w26 27f93943
w27 6a94f767	w28 c0a69407	w29 d19da4e1
w30 ec1786eb	w31 6fa64971	w32 485f7032
w33 22cb8755	w34 e26d1352	w35 33f0b7b3
w36 40beeb28	w37 2f18a259	w38 6747d26b
w39 458c553e	w40 a7e1466c	w41 9411f1df
w42 821f750a	w43 ad07d753	w44 ca400538
w45 8fcc5006	w46 282d166a	w47 bc3ce7b5
w48 e98ba06f	w49 448c773c	w50 8ecc7204
w51 01002202		

A.3 Expansion of a 256-bit Key

Key =

60 3d eb 10 15 ca 71 be 2b 73 ae f0 85 7d 77 81
1f 35 2c 07 3b 61 08 d7 2d 98 10 a3 09 14 df f4

Nk = 8

w0 = 603deb10 w1 = 15ca71be w2 = 2b73aef0 w3 = 857d7781

w4 = 1f352c07 w5 = 3b6108d7 w6 = 2d9810a3 w7 = 0914dff4

AES-256에는 추가 규칙이 있다. $i \bmod 8 = 4$ 인 위치에서는 ROTWORD/Rcon 없이 SUBWORD()만 추가 적용한다.

w08 9ba35411	w09 8e6925af	w10 a51a8b5f	w11 2067fcde
w12 a8b09c1a	w13 93d194cd	w14 be49846e	w15 b75d5b9a
w16 d59aecb8	w17 5bf3c917	w18 fee94248	w19 de8ebe96
w20 b5a9328a	w21 2678a647	w22 98312229	w23 2f6c79b3
w24 812c81ad	w25 dadf48ba	w26 24360af2	w27 fab8b464
w28 98c5bfc9	w29 bebd198e	w30 268c3ba7	w31 09e04214
w32 68007bac	w33 b2df3316	w34 96e939e4	w35 6c518d80
w36 c814e204	w37 76a9fb8a	w38 5025c02d	w39 59c58239
w40 de136967	w41 6ccc5a71	w42 fa256395	w43 9674ee15
w44 5886ca5d	w45 2e2f31d7	w46 7e0af1fa	w47 27cf73c3
w48 749c47ab	w49 18501dda	w50 e2757e4f	w51 7401905a
w52 cafaaae3	w53 e4d59b34	w54 9adf6ace	w55 bd10190d
w56 fe4890d1	w57 e6188d0b	w58 046df344	w59 706c631e

Appendix B - Cipher Example

원문은 AES-128 한 block 이 각 round 를 지나면서 State 가 어떻게 변하는지 intermediate value 를 전부 보여준다.

Input =
32 43 f6 a8 88 5a 30 8d 31 31 98 a2 e0 37 07 34

Key =
2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c

초기 State 배열:

32 88 31 e0
43 5a 31 37
f6 30 98 07
a8 8d a2 34

초기 Round Key:

2b 28 ab 09
7e ae f7 cf
15 d2 15 4f
16 a6 88 3c

10 round 가 완료된 최종 output 은 다음과 같다.

39 02 dc 19
25 dc 11 6a
84 09 85 0b
1d fb 97 32

이를 AES byte sequence 순서로 다시 쓰면:

39 25 84 1d 02 dc 09 fb dc 11 85 97 19 6a 0b 32

학습 주석 | 이 Appendix B 는 구현 검증에서 매우 중요합니다. 최종 ciphertext 만 비교하면 오류가 어느 round 에서 생겼는지 알기 어렵지만, intermediate State 를 비교하면 SubBytes/ShiftRows/MixColumns/AddRoundKey 중 어디서 틀렸는지 바로 좁힐 수 있습니다.

Appendix C - Example Vectors

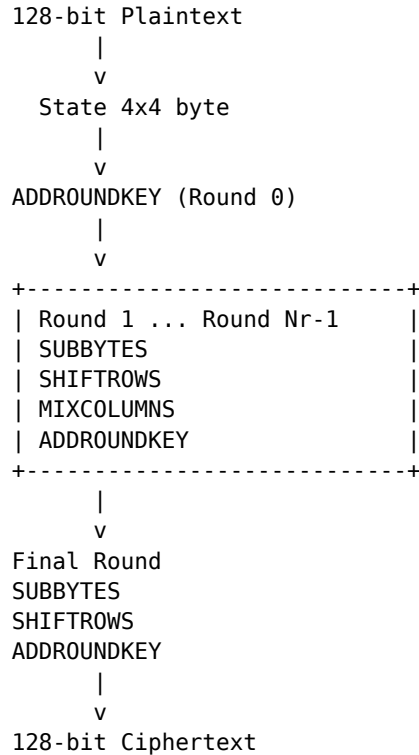
NIST Computer Security Resource Center 는 AES-128, AES-192, AES-256 에 대해 intermediate value 가 포함된 상세 example vector 를 별도로 제공한다. 2023 update 에서는 기존 Appendix C 의 예제 본문 대신 해당 NIST reference 로 연결하도록 정리되었다.

Appendix D - Change Log (Informative)

16. 출판물의 formatting 을 개선하고 문장을 명확하게 수정했다.
17. Title page, Foreword, Abstract, Keywords 를 front matter 에 추가했다.
18. 법령, 규정, 표준, validation program 관련 공표 내용을 최신화했다.
19. Section 1 에 AES 개발 관련 reference 를 추가·갱신하고 AES-128/192/256 을 명시했다.
20. 기존 Section 2.2 를 List of Functions 와 Algorithm Parameters and Symbols 로 분리했다.
21. 용어, 함수, 기호 목록을 더 포괄적으로 정리했다.
22. Byte sequence indexing 규칙을 Section 3 에서 더 명확하게 재구성했다.
23. Polynomial 해석을 Section 4 로 이동하고 관련 표를 수정했다.
24. State/Word index 설명을 개선했다.
25. Finite field 참고문헌을 추가하고 $GF(2^8)$ 관련 heading 을 명확하게 바꿨다.
26. $GF(2^8)$ multiplication 설명을 일반화하고 XTIMES 예제를 강화했다.
27. Fixed matrix multiplication 설명을 단순화하고 MixColumns 관련 설명을 연결했다.
28. Multiplicative inverse 설명을 새 Section 4.4 로 이동했다.
29. Section 5 의 algorithm component 관계 설명을 확대했다.
30. CIPHER, KEYEXPANSION, INVCIPHER pseudocode formatting 을 개선했다.
31. SHIFTRROWS/INVSHIFTRROWS 설명을 개선했고 기존 inverse 설명의 오류를 수정했다.
32. AES-128/192/256 KEYEXPANSION illustration 을 새로 추가했다.
33. Equivalent inverse cipher 의 modified key expansion algorithm 을 별도 pseudocode 로 추가했다.
34. Key generation reference 를 NIST SP 800-133 Rev.2 로 갱신했다.
35. Implementation attack 관련 설명을 확대했다.
36. References 를 업데이트하고 철회된 publication 을 교체했다.
37. Appendix C 의 고정 예제 대신 NIST 가 유지관리하는 example vector reference 를 사용하도록 변경했다.
38. Appendix D 를 새로 만들어 이번 update 변경사항을 정리했다.

학습 주석 | 2023 update 는 AES 의 핵심 cryptographic algorithm 자체를 바꾼 것이 아니라 표현, 설명, 그림, reference 및 구현 보안 관련 설명을 정비한 업데이트입니다.

AES 전체 흐름 한 장 정리



동시에 원래 AES Key 는 KEYEXPANSION()을 통해 Round Key 들로 확장되어 각 ADDROUNDKEY()에 공급된다.
Decrypt 방향에서는 inverse transformation 을 역순으로 적용하여 원래 State 를 복원한다.

이 문서에서 반드시 기억할 12 가지

39. AES block size 는 항상 128 bit 다.
40. AES-128/192/256 의 숫자는 key length 다.
41. AES-128/192/256 의 round 수는 10/12/14 다.
42. State 는 4x4 byte 배열이며 input 은 column 방향으로 배치된다.
43. SUBBYTES 는 S-box 를 사용하는 non-linear byte substitution 이다.
44. SHIFTRows 는 row 마다 다른 크기의 cyclic shift 다.
45. MIXCOLUMNS 는 GF(2⁸) fixed matrix multiplication 이다.
46. ADDROUNDKEY 는 XOR 이다.
47. Final round 에는 MIXCOLUMNS 가 없다.
48. KEYEXPANSION 은 원래 key 를 Nr+1 개의 Round Key 로 확장한다.
49. INVERSE 는 원래 변환을 되돌리는 역변환이다.
50. 실제 구현은 NIST example vector 와 intermediate State 를 bit-exact 로 비교해야 한다.

주요 English 용어 빠른 사전

English	한국어	의미
plaintext	평문	암호화 전 데이터
ciphertext	암호문	암호화 후 데이터
encrypt / encipher	암호화	plaintext -> ciphertext
decrypt / decipher	복호화	ciphertext -> plaintext
inverse	역변환/역연산	변환 결과를 원래 값으로 되돌리는 반대 연산
state	상태 배열	AES 중간 16-byte 값
round	라운드	반복되는 변환 단계

round key	라운드 키	각 round 에서 XOR 되는 128-bit key
key schedule	키 스케줄	전체 round key sequence
S-box	치환표	byte 를 다른 byte 로 변환하는 비선형 table
finite field	유한체	원소 개수가 유한한 수학적 field
Galois Field	갈루아 체	finite field 의 다른 표현
affine transformation	아핀 변환	matrix multiplication + vector addition
XOR	배타적 OR	같으면 0, 다르면 1
mode of operation	운용 모드	block cipher 를 실제 데이터 암호화 구조로 사용하는 방법